

Hephaestus

Autonomous Invention Harness · version 1.2

30 September 2026 · Implementation baseline. No invention runtime or scientific campaign has been validated by this package.

Reader edition: master specification and four normative supplements. The full obligation matrix, acceptance cases, schemas and recorded checks accompany this PDF in the package.

Contents

1. Product definition and success	3
2. Scope, boundaries, and first release	4
3. Non-negotiable scientific and execution invariants	4
4. Mission compiler and autonomy contract	5
5. End-to-end architecture	5
6. Knowledge acquisition and evidence graph	6
7. Discovery Engine: autonomous opportunity generation	6
8. Mechanism search and invention operators	7
9. Hypothesis Compiler	8
10. Quality, novelty, and prior-art attack	8
11. Search strategy and portfolio allocation	9
12. Adversarial review and evidence independence	9
13. Experiment Compiler and discrimination	10
14. Statistical validity and result semantics	10
15. Prototype realization and invention dossiers	11
16. Epistemic state and evidence lifecycle	11
17. Compiled micro- and nano-harness execution	12
18. Routing, Jev, models, and context economy	13
19. Durable scheduling, costs, and recovery	13
20. Security, confidentiality, and bounded autonomy	14
21. Data contracts and capability interfaces	14
22. Reference technology architecture and repository	15
23. CLI, TUI, gateway, and user control	16
24. Self-improvement without evaluator capture	16
25. Evaluation framework and proof of value	17
26. Operational targets and acceptance gates	17
27. Implementation sequence and release strategy	18
28. Worked example: context assembly research	19
29. Risks, open questions, and decision discipline	19
30. Source register and corrected research grounding	20
31. Final readiness contract	21
Appendix 1: Qualification and lineage	22
Appendix 2: First domain and campaign	25
Appendix 3: Retrieval and memory security	28
Appendix 4: Required autonomous self-improvement	29

Hephaestus — Autonomous Invention Harness

Master specification · version 1.2 · 30 September 2026

Document status: Implementation baseline for staged development. The architecture, contracts, evaluation plan, and reference validators are specified; the invention runtime is not implemented or experimentally validated by this package.

Project name: Hephaestus (hephaestus). This is the canonical project name for this specification. Tachyon is an optional execution integration, not a required dependency. This specification does not assume access to, compatibility with, or inspection of the current Tachyon repository.

Primary outcome: From an authorized broad goal, autonomously discover worthwhile problems, originate diverse mechanism candidates, compile high-quality testable hypotheses, run permitted experiments, and deliver reproducible invention dossiers that distinguish evidence from speculation. The runtime must also autonomously improve its own research and execution process through qualified, versioned changes within its authorization and resource envelope.

Core principle: Models propose; evidence updates beliefs; deterministic policy governs execution; independent verification governs promotion.

Reading and authority

This document is the human-readable normative specification. `requirements.json` provides stable requirement IDs and acceptance-test mappings. `schemas/contracts.schema.json` defines the included interchange records. `IMPLEMENTATION_PLAN.md` sequences the build. `ACCEPTANCE_TESTS.md` specifies future runtime tests. `HANDOFF.md` is the coding-agent entry point. `validation/REPORT.md` states which package checks actually ran. Version 1.2 includes normative supplements: `docs/QUALIFICATION_CONTRACT.md`, `docs/CONTEXT_ASSEMBLY_CAMPAIGN.md`, `docs/RETRIEVAL_AND_MEMORY.md`, `docs/SELF_IMPROVEMENT.md`, and the clause/obligation matrix `docs/OBLIGATIONS.md`. `release-scopes.json` separates contract readiness from runtime acceptance. These supplements refine the identified sections; unresolved conflicts remain blocking.

MUST and MUST NOT denote release-blocking obligations for the applicable milestone. SHOULD denotes a default that requires a recorded reason to depart from. MAY denotes an optional capability. Numeric settings labeled **provisional** are initial product or experiment-design choices, not measured performance or statistically justified sample sizes.

Authority order: approved safety and authorization policy; this specification and its requirements register; published schema contracts; approved architecture decisions; implementation plans; examples. A conflict between normative sources blocks the affected build task until an explicit versioned resolution. Examples never override a requirement.

External research is cited with source IDs such as [S01]. The references establish relevant precedents or technical semantics, not proof that this proposed system will work. All architecture choices not explicitly attributed to research are design proposals. The package does not claim independent expert-panel review.

1. Product definition and success

The product is a persistent, bounded, evidence-driven invention system, not an idea chatbot and not merely a coding agent. A user should be able to state a goal such as “Find better ways to reduce the cost of correct context assembly in coding agents,” authorize a resource envelope, and receive useful hypotheses without supplying the hypotheses, mechanisms, or research questions themselves.

The system must perform three distinct jobs. The **Discovery Engine** decides which problems deserve investigation. The **Hypothesis Genesis Engine** searches for mechanisms and derives observations that distinguish them from alternatives. The **Invention Engine** constructs and tests realizations, then assembles evidence and artifacts into a dossier. Their feedback loops remain connected: failed experiments can reveal new pressure points, and engineering failures can expose missing assumptions rather than refute a scientific claim.

The primary quality unit is an **evidence-qualified hypothesis**: a specific, useful, bounded proposal whose important premises are traced to sources or explicitly labeled assumptions, whose predictions are operationalized, and whose test could distinguish meaningful outcomes. A high-quality hypothesis can be false. Discovering that it is false through a valid,

informative experiment can be a successful research outcome.

A **validated solution** is a scoped, reproduced realization meeting its scientific, engineering, and guardrail gates; it can be a useful known solution and must display its rediscovery status. The longer-term product outcome is a **validated invention candidate**: an implemented realization that meets its predeclared engineering criteria, has supporting evidence within declared conditions, survives the prescribed verification process, and has a protected claim-level difference assessment with a verified prior-art claim chart. Neither label is automatically a globally novel invention, a patentable invention, a commercially viable product, or a universally correct theory.

Optimize verified information and useful outcomes per unit of constrained resources. Do not optimize idea count, tokens generated, number of agents, self-rated cleverness, or a product of arbitrary novelty and feasibility scores. No general claim of superiority is permitted until the matched-baseline evaluation in Section 25 passes.

2. Scope, boundaries, and first release

The first release targets computational domains with inspectable artifacts, affordable repeatable tests, and enforceable isolation: software systems, algorithms, developer tools, retrieval strategies, scheduling policies, and agent infrastructure. A first successful application is deliberately narrower than a universal autonomous laboratory.

The initial product supports natural-language mission intake, approved corpus ingestion, autonomous opportunity discovery, mechanism generation, hypothesis compilation, prior-art investigation, experiment planning, sandboxed code experiments, an evidence archive, and dossier export. It includes a local CLI, a conversational TUI, and a resumable gateway protocol. The UI must expose real state and evidence, not invent a narrative of agents working.

Domain packs later add symbolic mathematics, numerical simulations, CAD, manufacturing constraints, or laboratory integrations. Each pack supplies units, validity conditions, trusted evaluators, experiment templates, instrument capability descriptions, and risk policies. A domain pack supplies the language and tools of investigation; it must not quietly supply the target hypotheses in an “autonomous generation” evaluation.

Physical actions, biological or chemical work, live production changes, purchases, publications, patent filings, and external communications are not enabled by the initial execution profile. They require separate capability authorization and, where applicable, qualified human supervision. The system may stop at a useful experiment plan when an experiment cannot be performed safely or within budget.

The product must operate without a local GPU, graph database, particular model family, commercial search service, Jev, or Tachyon. Local GPU acceleration and specialized decision models are optional optimizations justified by measured workload benefit.

Explicit non-goals are unrestricted self-directed research, guaranteed invention novelty, automatic legal conclusions, unlimited recursive agents, replacement of all domain expertise, and manufacturing a positive result to satisfy a completion target.

3. Non-negotiable scientific and execution invariants

A source quotation, a measured observation, a model inference, an assumption, a mathematical derivation, and a human judgment are distinct record types. Converting one into another requires an explicit derivation or evidence operation; repetition does not increase evidential strength.

An absent graph edge does not mean a combination has never been tested. A failed search does not establish global novelty. Several papers using the same dataset or repeating the same claim are not independent replications. Several agents agreeing are not empirical corroboration.

Failure to reach a product target does not automatically refute the proposed mechanism. Failure to reject a statistical null does not establish that the mechanism is absent. A successful simulation supports a claim about the specified model and conditions, not automatic transfer to a physical system. A formal proof certifies the formal statement and assumptions actually encoded, not every interpretation of its prose description.

Deterministic schema validation establishes structure, not semantic truth. Deterministic execution of incorrect code is still incorrect. A Jev-style or other typed model output remains a probabilistic prediction; it is not a policy engine, proof checker,

or calibrated scientific posterior merely because its output has a fixed shape. TypeSafe describes Jev as returning typed probabilistic decisions; third-party OpenJev implementations must be assessed individually. [S07, S08]

A hypothesis without an accessible test may remain valuable in an exploratory archive. It cannot be labeled test-ready until it has an operationally credible discriminating test, or an explicit proof obligation for formal work. Budget exhaustion, missing instruments, and unavailable evidence produce blocked or inconclusive states, never success.

The runtime must preserve all outcome classes, negative results, failed approaches, and superseded versions. It must not change confirmatory endpoints after seeing outcomes and then present the new endpoints as predeclared. It must not broaden an authorization grant, spend beyond a reserved envelope, or promote itself based solely on self-evaluation.

4. Mission compiler and autonomy contract

Input is a broad goal plus an existing or newly approved authorization profile. The Mission Compiler produces a versioned Mission: intended beneficiary, objective, domain boundaries, practical constraints, resource envelope, permitted tools and destinations, forbidden actions, confidentiality, success metrics, evidence standard, and stop conditions.

The compiler must identify ambiguities that could materially change the mission. It may resolve reversible choices with explicit assumptions: terminology, initial search vocabulary, initial corpus, or a tentative subdomain. It must request authorization for unapproved spending, external disclosure, irreversible operations, or materially ambiguous risk. Within the approved workspace and policy, ordinary reversible research actions should proceed without repeated permission prompts.

A mission does not need a seed hypothesis. It does need a value frame. For a goal as broad as “invent something useful,” the system uses the owner's approved standing priorities and resource profile to propose a bounded portfolio. It must not substitute its own values, silently choose a sensitive application, or launch an unbounded search of everything.

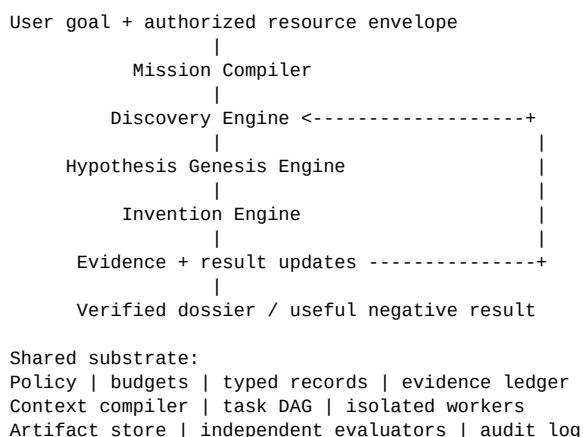
The compiled objective separates user intent from metrics. “Reduce latency” is not equivalent to “reduce output quality until the benchmark is fast.” Guardrails must cover correctness, workload scope, privacy, resource use, and unacceptable trade-offs. Each inferred requirement carries provenance and an override path.

Autonomy is permission-scoped rather than a single intelligence setting. Suggested profiles are **plan-only**, **local-research**, **sandbox-experiments**, and **supervised-external**. Local-research may use already authorized network research services; sandbox-experiments adds bounded code execution. No profile grants physical actuation or public disclosure implicitly.

Mission changes create a new version with an impact analysis. A changed goal, cost limit, dataset policy, or quality margin must invalidate dependent plans where relevant. A mission may pause without losing work; cancellation prevents new dispatches and initiates safe reconciliation of in-flight operations.

5. End-to-end architecture

The core is a small authoritative control plane with replaceable reasoning and execution adapters. Domain services are logically separated but may initially run in one local process. The design does not require microservices.



The Mission service owns intent and policy versions. Knowledge services own source acquisition, normalization, graph views, and provenance. Discovery owns opportunity candidates. Genesis owns mechanism search, hypothesis compilation,

and lineage. Experiment services own plan compilation, runner selection, raw data, and analysis. Verification services own readiness and promotion decisions. The scheduler owns dispatch, reservations, retries, and recovery, not scientific truth.

Models cannot directly change authoritative state. They return proposals conforming to a contract. The control plane validates the proposal, checks permissions and preconditions, records its provenance, and either accepts it as a proposal, dispatches an authorized operation, or rejects it with a reason code.

The evidence ledger is the system of record. Graph databases, embeddings, summaries, dashboards, and search indices are rebuildable views. No essential finding should exist only inside an agent conversation. Artifacts are immutable and content-addressed; mutable “latest” pointers resolve to explicit versions.

The normal loop is observe, formalize, discover, generate, compile, challenge, test, update, and reallocate. It is not a fixed waterfall: early prior-art checks prevent wasted work, inexpensive tests can precede extensive design, and unexpected results may create new opportunities.

6. Knowledge acquisition and evidence graph

Ingest authorized papers, technical documentation, patent documents, repositories, issue discussions, products, benchmark results, textbooks, datasets, and prior internal experiments. Acquisition adapters report coverage, pagination, inaccessible material, timestamps, licensing information when available, and retrieval failure. Partial access must remain visible.

Each source includes its locator, retrieval time, available publication or update date, content hash, source category, access scope, and parser version. Each extracted claim links to an exact source span or a reproducible data query. A title or search snippet is a discovery lead, not equivalent to reading the full source. Critical claims cannot be promoted on an inaccessible abstract alone when the missing methods or conditions determine validity.

The graph includes observations, claims, variables, conditions, mechanisms, constraints, assumptions, artifacts, experiments, failures, and opportunities. Relations include supports, contradicts, requires, enables, fails-under, derived-from, tests, supersedes, equivalent-under, and possible-analogy. Hypothesized links are marked as such and never rendered as established causal edges.

Evidence records separate when an event or finding was valid from when the system learned of it. This supports time-scoped prior-art reconstruction and later corrections. Retraction, contradiction, or corrected data creates a new event and flags dependent conclusions for review. It does not erase the prior record.

An evidence dependency graph groups common datasets, labs, codebases, citations, or shared derivations to avoid double-counting. Deduplication requires more than text similarity. Where origin cannot be determined, independence is unknown.

Start with relational records, full-text search, and explicit typed edges. Add vector or graph-specific stores behind interfaces only when retrieval evaluations demonstrate benefit. Graph2Idea is a relevant graph-context precedent, not proof that a graph is required or universally superior. [S05] Provenance export should support entity, activity, and actor relationships compatible with the concepts in W3C PROV. [S09]

Retrieval qualification [R-106, R-107]. Source-span provenance does not establish retrieval completeness. Qualify deep target discovery, wide relevant-set collection, contradiction retrieval and known near-match detection against independently adjudicated scoped fixtures. Preserve captured bytes, parser transformations and stable span coordinates; report missing access separately. docs/RETRIEVAL_AND_MEMORY.md defines the required metrics and cases. [S13]

7. Discovery Engine: autonomous opportunity generation

The Discovery Engine produces structured pressure points, not finished invention ideas. Every opportunity must explain who benefits, what currently limits them, what evidence indicates the limitation, why the limitation may be addressable, and what would be learned by investigating it.

Its operators include bottleneck mining from profiles and traces; contradiction mining from competing objectives; recurring failure analysis; anomaly detection; unmet-need extraction from authorized user evidence; cross-domain structural matching; capability-unlock detection; assumption mining; and reactivation of earlier failures whose boundary conditions have changed.

An anomaly requires an uncertainty-aware discrepancy between a prediction and observation, not two isolated numbers from different conditions. A trade-off requires evidence that the variables are coupled in the relevant design regime. A newly available capability requires verified availability and a concrete account of which previous constraint it changes. A claimed customer need must distinguish an actual observation from a simulated customer persona.

Opportunity candidates are challenged against alternative explanations: poor implementation, measurement error, already-solved problems, scope mismatch, unavailable resources, or a trivial known optimization. A rediscovered known solution may be useful, but it must be labeled a rediscovery rather than an invention opportunity of established novelty.

The output is an opportunity with problem statement, beneficiary, context, pressure-point type, evidence references, suspected bottleneck, causal uncertainty, value estimate, feasibility envelope, initial prior-art query plan, and unanswered questions. Unknown evidence is explicit. Candidates can be created from conjecture but remain speculative until basic grounding is obtained.

Discovery evaluates its own yield separately from hypothesis quality. It tracks the fraction of proposed opportunities that correspond to real, material, addressable needs under blind review or objective measurements. Generating excellent hypotheses for nonexistent problems is not success.

A portion of discovery effort is reserved for less conventional directions and for auditing discarded opportunities. This reduces reliance on whatever the current retrieval system or evaluator already considers familiar and plausible.

8. Mechanism search and invention operators

The Genesis Engine searches over structured mechanisms. A mechanism record states entities, relevant variables, causal or computational relationships, prerequisites, expected effects, operating regime, failure modes, and a minimal realization. “Use AI,” “add a graph,” or “make it adaptive” is not a mechanism.

Operators are versioned transformations with input contracts, applicability checks, bounded outputs, and provenance. Initial operators are abduction, contradiction resolution, assumption removal, assumption inversion, mechanism substitution, structural analogy transfer, composition, subtraction, scale or time transfer, failure resurrection, and capability unlock.

Abduction proposes explanations for an observation and must enumerate at least one competing explanation. Contradiction resolution searches for decoupling through spatial separation, temporal separation, different control variables, or a substituted mechanism. Assumption removal asks whether a constraint is fundamental, conventional, or an artifact of the current implementation.

Analogy transfer must map relational structure, not just vocabulary. Its output includes a source mechanism, target entities, preserved relations, broken relations, and boundary conditions that could invalidate the transfer. A biological metaphor without an executable or physical mapping remains an analogy, not an engineering design.

Composition checks interface compatibility, units, resource budgets, and interactions. Subtraction tests whether a component's required function disappears, moves elsewhere, or was never necessary. Scale transfer must account for regime changes; the same qualitative mechanism need not behave similarly at different spatial or temporal scales.

Hard filters are restricted to justified constraints under declared assumptions: malformed records, contradictory requirements, type or unit errors, exceeded resource bounds, exact duplicates, or formally checked impossibility in the specified model. Plausibility models may prioritize or flag candidates, but cannot irreversibly eliminate every unusual proposal. Proposed changes to accepted scientific assumptions require explicit labeling and stronger tests, not silent rejection solely because they are unfamiliar.

No fixed number of “genius agents” is required. Diversity comes from operators, evidence paths, assumptions, search branches, and evaluation methods. Model diversity is optional and must be tracked rather than confused with independence.

9. Hypothesis Compiler

The compiler transforms a mechanism candidate into an operational hypothesis. Its output includes context, intervention, comparator, proposed mechanism, measurable predictions, estimands, meaningful effect bounds, boundary conditions, competing explanations, required observations, analysis requirements, and explicit outcomes that would count against the claim.

The compiler separates **mechanistic claims** from **engineering targets**. A mechanism may reduce repeated work yet fail to deliver a useful end-to-end speedup because bookkeeping costs dominate. Conversely, a speedup may meet the product target for reasons unrelated to the claimed mechanism. These require different conclusions and often different tests.

For empirical claims, a prediction must identify the measured quantity, unit, direction, observation scope, uncertainty treatment, and decision rule. For formal claims, it must define a proposition, assumptions, proof or counterexample procedure, and a trusted checker. Purely subjective preferences require an appropriate human evaluation protocol; a simulated user does not substitute for an observed user.

A compiler check asks: could plausible data support, contradict, or leave this claim unresolved? Are the test's outcomes meaningfully different under the proposed and competing mechanisms? Are key variables measurable? Is the comparator relevant? Are the preconditions realizable within the permitted environment? Does the hypothesis merely restate its own success metric?

A test-ready hypothesis requires an accessible discriminating test. Exploratory concepts without one remain `EXPLORATORY` or `BLOCKED_TESTABILITY`, preserving their potential value without mislabeling their readiness. The rule is therefore “no credible discriminator, no test-ready hypothesis,” not “delete every idea that cannot be tested today.”

The compiler must not invent numerical thresholds or sample sizes and present them as empirically justified. A threshold can come from a user requirement, deployment economics, a prior study, a pilot, or an explicitly provisional design choice. The provenance of that choice is mandatory.

Revision creates a new hypothesis version with lineage. A changed mechanism, boundary condition, primary endpoint, or falsifier cannot inherit confirmatory support without an explicit assessment of whether the old evidence still applies.

10. Quality, novelty, and prior-art attack

High quality is established by gates plus a multi-objective profile. Mandatory gates cover relevance, traceability, operational clarity, internal consistency, discriminating testability, authorization, and honest uncertainty. The profile retains usefulness, estimated feasibility, novelty evidence, information value, test cost, test latency, mechanistic specificity, and diversity contribution.

These dimensions must not be collapsed into a supposedly objective universal score. Scheduling may use a versioned utility policy after hard gates, but the underlying dimensions and their uncertainty remain inspectable. A cheap, unusual hypothesis can coexist with a conventional one with higher estimated deployment value.

The prior-art process decomposes the candidate into atomic claims and relationships: components, combination, mechanism, use context, operating regime, and claimed result. It searches synonyms, equivalent formulations, adjacent domains, citations, code implementations, products, and relevant patent documents where authorized.

Search proceeds in two stages. A fast pass catches clear rediscoveries before costly experiments. A deeper claim-level comparison is required before any externally facing novelty statement. Reports include search dates, databases or corpora covered, query families, retrieved near-matches, missing access, relevant passages, and a specific account of similarities and differences.

Allowed conclusions are `KNOWN`, `NEAR_MATCH`, `NO_MATCH_WITHIN_SEARCH_SCOPE`, `CONFLICTING`, and `UNRESOLVED`. None means guaranteed global novelty. The absence of a paper in an index is not evidence that the mechanism has never existed. The system must not automatically issue patentability, inventorship, freedom-to-operate, or legal ownership conclusions.

Potentially valuable proprietary mechanisms need a confidentiality-aware search plan. Querying an external service can itself reveal an idea; a permitted literature-search tool does not necessarily authorize submission of the full confidential mechanism. Redacted searches and explicit disclosure approval are separate operations.

A new relationship between known components may be worthy of investigation. A relabeling of an established mechanism is not made novel by changing terminology. The difference must be shown with a claim chart, not an LLM's numerical impression.

Prior-art labels [R-112]. Known useful mechanisms may qualify as `validated_solution` with explicit rediscovery labeling. The `validated_candidate` label additionally requires a verified claim chart and protected assessment of specific differences; neither `near_match` nor `scoped no-match` by itself establishes novelty. Scientific status is unaffected by novelty labeling.

11. Search strategy and portfolio allocation

Search is an explicit graph of opportunities, mechanisms, hypotheses, refinements, and experiments. Nodes store their parents, operator, evidence snapshot, evaluated outcomes, estimated costs, and rejection reasons. Equivalent candidates may share evidence but keep lineage where conditions differ.

The first implementation uses an interpretable bounded beam or best-first search with a diversity archive. Monte Carlo tree search, quality-diversity search, Bayesian optimization, or bandit policies are interchangeable extensions, not prerequisites. MC-NEST provides a relevant search-based hypothesis-generation precedent; it does not establish that its specific search rule is optimal for this harness. [S04]

A resource allocation policy must preserve both exploitation and exploration. A provisional starting allocation is 55% to promising branches, 25% to diverse or uncertain branches, 15% to replication and counterevidence, and 5% to auditing rejected candidates. These percentages apply to a specified remaining discretionary compute budget, exclude mandatory security and integrity checks, and may change only through a logged policy version.

Expected information gain is computed only when a defensible predictive model is available. In that case, the scheduler can estimate the expected reduction in uncertainty about competing hypotheses. Where probabilities are unjustified, use qualitative discriminability, scenario bounds, and explicit unknowns instead of fabricating a Bayesian posterior.

A possible action priority is expected decision improvement divided by estimated remaining cost, with separate wall-clock and resource constraints. This is a scheduling heuristic, not an assertion that every form of discovery can be reduced to money. Mission-specific value weights belong to the owner and remain visible.

The scheduler imposes bounds on expansion depth, duplicate generation, concurrent workers, model calls, retrieval volume, elapsed budget, and total candidate count. It detects repeated paraphrases and unproductive refinement loops. A branch stops when it is dominated under the current mission, invalid, blocked, sufficiently investigated, or not worth its next authorized experiment.

A mission can end with no validated invention. Its deliverable must then explain what was learned, which opportunities remain, what evidence was missing, and why further work was not justified within the authorized envelope.

12. Adversarial review and evidence independence

Review roles correspond to distinct failure questions. Scientific review challenges causal logic and boundary assumptions. Domain engineering review checks realizability. Statistical review checks measurement and inference. Prior-art review looks for rediscovery. Reproducibility review attacks hidden dependencies. Security review attacks tool and data boundaries. User-value review tests whether the proposed improvement matters.

The runtime selects roles based on the candidate, not a decorative fixed-size swarm. A software hypothesis does not require a manufacturing reviewer unless manufacturing constraints actually apply. Reviewers receive role-specific evidence and checks, not just a common prompt with different job titles.

Candidate generation, analysis execution, and promotion are separated by permissions and artifacts. Where practical, a reviewer sees a blinded candidate without its generator's enthusiasm, model identity, or self-rating. Reviewers report concrete objections with evidence, severity, affected claims, and a proposed resolution or test.

Voting does not establish truth. A deterministic verification failure can block promotion despite unanimous model approval. A speculative objection does not automatically defeat a claim backed by stronger evidence. Disagreements remain in the dossier with their provenance and disposition.

Repeated reviews by the same model family and shared context are not described as independent experts. Model diversity, source diversity, and empirical replication are reported separately. A second evaluator using the same flawed measurement pipeline is not sufficient independence.

The expert-review workflow must support actual humans where stakes, domain limitations, or unresolved methodology warrant them. Human approval authorizes an action or accepts a decision; it does not transform unsupported claims into verified evidence.

Google's AI co-scientist uses specialized generation and review roles and reports both automated and expert evaluation. Its reported experimental settings involved expert guidance. This specification borrows role separation, not a claim that internal debate alone validates an invention. [S01]

13. Experiment Compiler and discrimination

The Experiment Compiler chooses the cheapest authorized experiment that can resolve the relevant uncertainty, not merely the cheapest activity associated with a hypothesis. It can produce a source check, counterexample search, analytical derivation, code experiment, simulation, prototype test, or a supervised external protocol.

Every `ExperimentPlan` binds the hypothesis version, claim IDs, operating conditions, comparator, intervention artifact, measurement procedure, units, primary endpoints, guardrails, sampling unit, analysis specification, stopping rule, evaluator hash, resource limit, and authorization scope. These are registered immutably before confirmatory results are accessed.

Competing explanations are explicit. For a proposed speed improvement, alternatives may include a warmed cache, dropped work, a weaker baseline, a changed dataset, or lower-quality outputs. A mechanism-sensitive ablation should distinguish the claimed causal path from those explanations.

A **discrimination matrix** maps predicted observations to the proposed mechanism, the strongest relevant alternative, and plausible artifacts or null behavior. Exact probabilities are optional and must not be invented. If the candidate and alternative predict indistinguishable outcomes in the proposed test, that test cannot establish the mechanism's distinct contribution.

Plans include positive or known-working controls where appropriate, negative controls, workload randomization, repeatability checks, environmental capture, and a plan for missing or corrupted observations. A control failure invalidates the affected inference until resolved.

Multi-fidelity experiments may progress from cheap analytical bounds to synthetic tests, representative workloads, and external deployment. Early tests can eliminate obvious failures, but successful lower-fidelity results do not bypass the validation needed for a higher-fidelity claim. The discrepancy between simulation and deployment must be measured or explicitly unresolved.

If a test is unavailable, the compiler returns a precise blocker: missing instrument, inaccessible dataset, unvalidated simulator, inadequate power, forbidden action, or resource overrun. It must not replace a physical experiment with an LLM's imagined outcome.

14. Statistical validity and result semantics

The default confirmatory path uses a predeclared fixed-sample analysis with a named estimand and appropriate uncertainty interval. Sample size follows a precision or power analysis using relevant variance information, or a conservative bound. A pilot used to choose the design belongs to exploration unless a valid combined analysis was planned in advance.

Sequential testing is optional and requires an approved method with documented assumptions. Repeatedly computing an ordinary fixed-sample test and stopping when it looks favorable is prohibited. Time-uniform confidence sequences are one possible method family, but their assumptions still need to match the experiment. [S10]

The system maintains an experiment-family ledger covering related hypotheses, endpoints, adaptive selection, and repeated looks. Confirmatory claims require a predeclared multiplicity strategy appropriate to the family. Exploratory screening may be useful without confirmatory guarantees, but must be labeled accordingly. POPPER is a relevant precedent for agentic hypothesis testing with an explicit statistical control framework; this design does not inherit its guarantees merely by implementing a similar loop. [S03]

Use the correct unit of analysis. Repeated timings inside the same repository, machine, dataset, or agent trajectory may be correlated. Pairing and clustering must follow the actual experimental design. The plan defines exclusions, missing-data treatment, precision requirements, and sensitivity analyses before confirmation.

For a claimed benefit of at least threshold θ , an interval entirely above θ can support that scoped target under the chosen method; an interval entirely below θ can contradict that target; overlap remains inconclusive. Mechanistic support is assessed separately. For an allowed quality loss m , noninferiority requires an appropriate lower bound on the quality difference exceeding $-m$, not merely a nonsignificant difference.

A result has three separate dimensions. **Execution validity:** valid, invalid, incomplete. **Scientific conclusion:** supported, contradicted, inconclusive, not assessed. **Engineering target:** met, not met, inconclusive, not assessed. The interpretation must identify the claim and conditions to which each conclusion applies.

No run may report “proven true” from a finite empirical test. All raw data, exclusions, analysis code, confidence or credibility method, model assumptions, and deviations are preserved. A result lacking enough information for an independent interpretation cannot pass a reproducibility gate.

Family and access contracts [R-100, R-101]. Persist typed experiment-family membership, selection lineage, error allocation, data-partition identity and broker-authenticated holdout access. A string naming a correction is not a qualification. The first campaign uses a bounded fixed Bonferroni family with feedback sealed until campaign closure; repeated campaigns or renamed families cannot reset exposure to the same partition. Retire used confirmation sets and require fresh independent workloads unless a separately qualified reuse protocol applies. The protected method registry binds assumptions, design and implementation hash; model assertions cannot qualify a method. `docs/QUALIFICATION_CONTRACT.md` defines the gates.

15. Prototype realization and invention dossiers

The Invention Engine translates a surviving hypothesis into the smallest realization that can establish its relevant claims. It should not build a complete product before determining whether the proposed mechanism contributes anything.

A prototype contract specifies functional behavior, interfaces, observable failure modes, constraints, dependencies, resource limits, and the exact claims it can test. Implementation workers operate in isolated workspaces and return artifacts plus build and test receipts. Prototype authors cannot silently alter protected evaluators or the baseline.

A component passing local tests is not sufficient for composition. Assembly verifies interfaces, version compatibility, global invariants, resource aggregation, interaction effects, and the end-to-end mission guardrails. An optimization that shifts cost to another stage must be accounted for in total system cost.

Successful prototypes advance through scoped validation: local correctness, representative experiments, clean-environment reproduction, held-out confirmation, and any required domain-specific verification. Production adoption, physical manufacturing, or publication is a separate authorized decision.

The dossier contains the problem and beneficiary, prior-art comparison, mechanism explanation, complete hypothesis versions, predeclared tests, artifacts and environment, raw data, analysis, results, counterevidence, failure history, uncertainty, scope limits, cost ledger, reproducibility commands, unresolved risks, and next justified action.

A negative-result dossier is a first-class output. It distinguishes an invalid test, a failed implementation, an unsupported mechanism, an uncompetitive engineering realization, and a resource-limited investigation. It identifies reusable knowledge without overstating generality.

AlphaEvolve illustrates the value of code generation connected to automated evaluators and an evolutionary search archive in computational domains. It is a precedent for the proposed software-first validation loop, not evidence that arbitrary inventions have reliable automatic evaluators. [S02]

16. Epistemic state and evidence lifecycle

Use explicit states rather than one continuously increasing confidence score. Opportunities move through discovered, grounded, prioritized, explored, parked, and closed. Hypotheses move through exploratory, compiled, reviewed, test-ready, testing, and assessed, with blocked and archived states available from relevant stages.

Scientific conclusion, engineering attainment, novelty assessment, and execution status remain orthogonal fields. A candidate may be scientifically supported but commercially unhelpful, mechanically inconclusive but faster in one benchmark, or useful but already known.

A promotion service checks evidence and invariants. Test-ready requires a valid hypothesis, credible discrimination plan, authorized executable route, cost reservation eligibility, and resolved blocking objections. A validated-candidate label additionally requires the configured confirmatory and reproduction gates plus a scoped prior-art assessment.

Evidence updates are append-only. Corrected observations, retractions, version changes, and newly found prior art can downgrade a conclusion. The service traverses dependent claims, marks affected dossiers stale, and queues re-evaluation where permitted. It must not automatically rerun expensive or externally consequential work without budget and permission.

Bayesian estimates are allowed only with explicit priors, likelihoods, model assumptions, and a method for accounting for dependent evidence. Uncalibrated model probabilities are stored as model judgments, not scientific posterior probabilities. Unknown uncertainty is represented as unknown rather than a narrow invented distribution.

Failure records retain operating conditions. A previously failed mechanism may be reconsidered when a material, tool, workload, or constraint changes. Reactivation creates a new version referencing the original failure and the evidence for the changed condition. It does not erase the failure or assume that the new condition fixes it.

An evidence snapshot is a reproducible, versioned selection of records and artifacts. Every hypothesis, experiment plan, model request, and dossier binds to such a snapshot so later knowledge does not silently alter the historical interpretation.

Promotion and current applicability [R-094–R-099, R-114]. Promotion MUST resolve the complete version-bound mission → opportunity → mechanism → hypothesis → plan → result → dossier chain. It MUST verify authenticated execution/analysis/reproduction/promotion assessments, primary endpoint and guardrail coverage, controls, units, accessible artifact bytes, applicable registration, qualified methods, current policy and nonstale dependencies. Missing trust or applicability blocks promotion. A historical result remains intact while current labels may become stale. `docs/QUALIFICATION_CONTRACT.md` supplies binding, invalidation and state-transition tables. The reference checks enforce selected metadata and fail closed without externally supplied trust; they do not authenticate real receipts or establish scientific truth.

17. Compiled micro- and nano-harness execution

The hierarchy is mission, opportunity investigation, research task, micro-task, and optional nano-task. It is a decomposition vocabulary, not a requirement to split every job into five levels. A nano-task is a small bounded operation, not necessarily a small language-model agent.

Compilation produces a typed `TaskSpec` and execution DAG. It selects tools, input artifacts, read and write contracts, required capabilities, budget bounds, timeout, retry policy, verification rule, isolation profile, and output schema. It does not generate and compile a new native application for every tiny operation.

A task should split only when child outputs have meaningful contracts, dependencies are known, independent work can proceed safely, and expected verification-adjusted benefit exceeds scheduling and coordination overhead. Coalesce tiny deterministic operations into one worker when that is faster. Preserve whole-problem context where decomposition would destroy causal or semantic coherence.

A typical opportunity can launch independent literature queries, unit checks, alternative-mechanism searches, and baseline inspections in parallel. A dependent experiment cannot run until its artifact, protocol, authorization, and budget prerequisites are satisfied. Parent verification includes cross-child conflicts and integration tests, not just a count of successful children.

Parallel workers read immutable snapshots and write isolated artifacts. They may propose edits but cannot concurrently mutate authoritative shared state. A reducer validates output schemas, checks declared effects, resolves explicit conflicts, and records accepted results in a deterministic order.

The scope of determinism is precise: given identical normalized inputs, recorded external responses, policy versions, and deterministic tools, orchestration replay should reproduce the same decisions. New model calls, network content, GPU numerical kernels, timing measurements, and physical experiments may not reproduce bit-for-bit. Store their outputs and environmental conditions rather than claiming universal determinism.

Unknown effects, undeclared writes, or unclear dependencies prevent speculative parallel execution. Serialize or require a stronger isolation contract. A semantic conflict detector supplied by a model may flag risk but cannot guarantee the absence of conflicts.

18. Routing, Jev, models, and context economy

The router chooses the cheapest sufficient authorized method under required reliability, latency, and resource constraints. This is a capability-and-risk decision, not a fixed staircase where every task must pass through every tool class.

Exact lookups, schema checks, arithmetic, hashes, unit conversions, and policy decisions belong to deterministic code. Symbolic solvers and proof tools handle suitable formal tasks. Search and data queries retrieve evidence. Simulators estimate behavior under explicit models. Typed semantic models may classify or prioritize ambiguous text. Generative models propose mechanisms, explain mappings, synthesize evidence, or draft experiment plans.

Jev and compatible implementations sit behind `DecisionProvider`. Their appropriate initial uses are relevance triage, candidate categorization, query routing, and advisory review prioritization. They must not certify novelty, mathematical validity, scientific truth, access control, budget approval, or a confirmatory result. Domain-specific calibration, abstention behavior, drift monitoring, and disagreement audits are required before any advisory decision is automated. [S07, S08]

No core interface contains a provider's model names, proprietary reasoning-level enums, SDK message types, or retrieval query language. Capabilities describe what is needed: structured proposals, bounded choices, image understanding, code generation, long context, tool use, cancellation, streaming, usage reporting, and data-handling guarantees. An adapter translates capabilities into provider-specific calls and rejects unsupported requirements.

The Context Compiler gives each task a bounded evidence packet: objective, exact inputs, relevant source spans, contradictions, inherited assumptions, contracts, and unresolved questions. It must preserve access to raw evidence through references. Summaries include provenance and cannot turn uncertain or conflicting findings into settled facts.

Cache keys bind to task semantics, schema, policy, source and artifact hashes, relevant prompts, model or tool version, and declared environmental dependencies. Cached results are invalidated on relevant changes. Cached evidence cannot be reused as an independent replication.

A local GPU is optional. The runtime must discover available capacity, respect a configured reserve for other work, and avoid forcing an unverified model to fit a 12 GB device. Unknown prices or uncertain capacity require an explicit estimate and a conservative bound before dispatch.

19. Durable scheduling, costs, and recovery

The scheduler is a bounded, fair, dependency-aware queue. It controls per-provider concurrency, memory, CPU, GPU, network, and experiment exclusivity. It prioritizes the critical path where appropriate without starving verification, exploration, or user interaction.

Every operation has a durable ID, task version, input digest, policy snapshot, cost reservation, attempt number, and lifecycle events. The operation is recorded before the effect is dispatched. Outputs are written to temporary content-addressed locations, validated, committed, and then reflected in authoritative state.

External side effects use idempotency keys where supported. If a worker crashes after a possibly completed operation but before receipt recording, the scheduler reconciles with the external system or marks the outcome unknown. It must not blindly retry a non-idempotent purchase, submission, publication, or physical action. The design does not claim universal exactly-once execution.

Budget accounting uses reserved, spent, released, and unresolved amounts in a defined currency. Concurrent operations reserve before execution. Provider usage is reconciled after completion. Token or time bounds are enforced independently where monetary metering is delayed. Price versions and exchange-rate provenance are recorded when conversion is used.

Cancellation prevents new reservations, signals cancellable work, preserves partial artifacts, and reconciles in-flight effects. Safe stopping may leave work in unknown or awaiting-reconciliation states. The UI must show this instead of claiming that everything stopped instantly.

Recovery replays append-only events to rebuild projections, validates checkpoints and artifact hashes, and requeues only operations whose retry semantics permit it. Fault-injection tests must cover crashes around reservation, dispatch, artifact write, commit, and acknowledgement boundaries.

The local MVP uses one authoritative writer and an embedded transactional database with content-addressed files. Distributed execution is a later adapter. Remote workers return receipts; they do not receive unrestricted database credentials. Concurrency must be measured against actual local limits rather than assuming that thousands of simultaneous agents improve throughput.

20. Security, confidentiality, and bounded autonomy

Retrieved papers, webpages, repository files, model outputs, and generated code are untrusted inputs. They cannot grant permissions, alter evaluator policy, request secret disclosure, or change the mission. Instructions embedded in a source remain source content.

Tool execution uses capability-scoped grants tied to the mission, operation, destination, and expiration. Workers receive only the data and credentials they need. Default filesystem access is a read-only input snapshot plus a dedicated writable output directory. Network access is denied unless explicitly required and authorized.

Generated code runs under an isolation profile appropriate to its risk. Process boundaries alone are insufficient as a security claim. The implementation must document sandbox limitations and test path traversal, symlinks, environment-variable leakage, subprocess escape, network exfiltration, dependency installation, and denial-of-service behavior.

Secrets live outside model-visible context and ordinary logs. Provider selection respects confidentiality and data-use policy. Sending private invention details to a service that may retain or train on them requires a compatible explicit authorization; a low price is not an authorization.

Protected evaluators, sealed benchmark data, policy files, and promotion rules are not writable by candidate-generation or prototype workers. Approval tokens bind to the exact operation and artifact hash. A later changed artifact requires a new approval where the grant no longer matches.

Risk checks occur at mission intake, task compilation, and just before execution. New tools or an evolved candidate may change the risk class. Unsupported hazardous experimentation is blocked or reduced to an appropriate non-actionable research summary, not routed around the policy through another agent.

Public exports require a disclosure check for secrets, restricted source material, private datasets, and unapproved invention details. A hash or internal timestamp is an audit aid, not a legal guarantee of ownership, priority, or protection.

Persistent poisoning and live containment [R-108–R-111]. Trust origin MUST survive summaries, caches, graph edges and cross-session memory. Quarantine propagates to dependent current artifacts and assessments; retrieved instructions cannot become behavioral policy. Protect owner-controlled memory writes. Verify effective sandbox configuration before each dispatch and monitor tested boundary violations during execution with fail-closed stopping and preserved receipts. Test revocation races, redirected/DNS/private-address egress, inherited descriptors and dependency hooks within the declared threat model. docs/RETRIEVAL_AND_MEMORY.md specifies qualification cases. [S14, S15]

21. Data contracts and capability interfaces

The included JSON Schema uses the 2020-12 dialect. It provides structural validation for mission, source, evidence, opportunity, mechanism, hypothesis, experiment plan, experiment result, task, decision, event, dossier, authorization grant, method qualification, experiment family, holdout access, verification receipt, and improvement candidate records. Schema conformance does not establish scientific adequacy; semantic validators and release tests cover cross-record rules. [S11]

Every domain record has a stable ID, schema version, record version, creation timestamp, and provenance context. References resolve to immutable record versions or artifact hashes. Units are explicit. Numeric unknowns are null with a reason, not zero. Confidence fields include the estimation method and calibration status where applicable.

The core interfaces are `ModelProvider`, `DecisionProvider`, `RetrievalProvider`, `EvidenceFetcher`, `KnowledgeStore`, `ArtifactStore`, `ExperimentRunner`, `AnalysisProvider`, `VerificationProvider`, `SandboxProvider`, `ExecutionBackend`, `PolicyEngine`, `BudgetLedger`, and `TraceExporter`.

All adapters expose capability discovery, health, version identity, request validation, cancellation semantics, error taxonomy, and cost or resource reporting where relevant. Typed errors distinguish unsupported capability, transient failure, authentication, quota, malformed output, evidence unavailability, policy denial, and unknown external effect.

`ModelProvider.propose` returns a proposal and usage receipt; it cannot mutate domain state. `DecisionProvider.evaluate` returns bounded choices, scores or probabilities, and calibration metadata; the control plane determines allowed use. `RetrievalProvider.search` returns provenance-bearing hits plus coverage and continuation information; it cannot claim exhaustiveness without a scoped contract.

`ExperimentRunner.run` returns raw artifacts and an execution receipt. `AnalysisProvider.analyze` applies a predeclared analysis specification and produces derived results. `VerificationProvider.assess` checks the applicable readiness or promotion gate with evidence references. These responsibilities may share a process initially but must not share unrestricted write permissions.

Adapters must be replaceable through contract tests. A provider migration must preserve stored evidence and task semantics, or clearly version any changed behavior. Unsupported capabilities cannot silently degrade into weaker behavior when they affect validity.

Version 1.2 contract migration [R-094, R-099–R-101, R-114]. Eighteen principal record types now include authorization grants, method qualifications, experiment families, holdout-access events, verification receipts and improvement candidates. References and trusted bindings are immutable; new plan/result/dossier fields make scope, guardrails and assessment chains explicit. The archived v1.0 and v1.1 schemas and examples remain historical compatibility inputs. Unsupported versions are rejected; any runtime migration preserves old bytes/hashes and records semantic differences instead of assigning new approval. Canonical registration uses the documented 1.1 hash profile and shared vectors.

22. Reference technology architecture and repository

Use a Rust control plane for policy, state transitions, scheduling, budgets, events, local gateway, and artifact manifests. Use Python subprocess workers for scientific computing, data analysis, solver integration, and domain-specific experiments. Use TypeScript for gateway clients and a later web UI; a native Rust TUI is an acceptable initial interactive interface.

This division is a proposed implementation choice, not a claim that Rust makes model inference or physical experiments faster. The important boundary is an auditable, typed, authoritative runtime separated from interchangeable reasoning and experiment tools. Reusing a verified Tachyon execution adapter is allowed when the real interface is inspected and contract tests pass.

Start as a modular monolith with isolated workers. Use an embedded transactional database for authoritative local state and a content-addressed filesystem store for artifacts. Full-text and typed relational queries are the minimum retrieval baseline. Graph traversal and vector search are optional adapters, not two additional mandatory infrastructure services.

The reference repository layout is:

<code>crates/domain</code>	typed records and invariants
<code>crates/policy</code>	capability and authorization checks
<code>crates/ledger</code>	events, budgets, projections
<code>crates/scheduler</code>	DAG, reservations, retries, recovery
<code>crates/gateway</code>	local API and event stream
<code>crates/cli</code>	commands and TUI
<code>services/discovery</code>	opportunities and pressure points
<code>services/genesis</code>	operators, search, compilation
<code>services/experiments</code>	plans, runners, result assembly
<code>workers/python</code>	sandboxed scientific workers
<code>adapters</code>	model, retrieval, Jev, Tachyon, stores
<code>schemas</code>	versioned interchange contracts
<code>evals</code>	fixtures, sealed suites, evaluators
<code>docs</code>	specifications and decisions

Provider SDK imports belong only in adapters. Domain modules do not know which search provider, model family, database engine, or harness backend is in use. Toolchain and dependency versions are pinned during bootstrap based on the implementation environment; this specification does not invent future package versions.

No distributed cluster, automatic local model installation, GPU runtime, or whole-world ontology is needed to complete the first vertical slice.

23. CLI, TUI, gateway, and user control

The main interaction is conversational but state-grounded. A user can give a goal, inspect the plan, approve a resource envelope, steer an investigation, compare hypotheses, request the evidence for a claim, pause work, or export a dossier. Every displayed status comes from stored events and current projections.

Reference CLI commands are `invent init`, `invent mission create`, `invent plan`, `invent run`, `invent attach`, `invent status`, `invent hypotheses`, `invent evidence`, `invent approve`, `invent pause`, `invent resume`, `invent cancel`, `invent export`, and `invent replay`. These are proposed product commands; they are not implemented by this specification package.

The TUI should show the mission, active investigations, current experiment, spend and reservations, blockers, material findings, and safe stop controls. It should default to concise progress and allow drilling into the exact hypothesis, source span, raw result, or rejection reason. It must never simulate activity by printing fictitious agent dialogue.

Gateway operations use stable IDs and idempotent request semantics where appropriate. Minimum endpoints cover missions, plans, tasks, hypotheses, evidence, approvals, artifacts, and an ordered event stream with resume cursors. Remote clients authenticate and obey the same policy; the gateway is not an alternate route around approval.

Steering changes future work through a new mission or plan version. A user may stop a branch or prioritize an opportunity without silently modifying the experiment that is already in confirmation. An explicit change invalidates or versions the affected confirmatory run.

A dossier view separately exposes scientific conclusion, engineering target status, novelty search status, and reproduction status. It must not compress these into a large green “invented” badge.

Long-running monitoring belongs to the installed runtime under an approved recurring mission. The presence of this capability in the specification does not mean this chat has scheduled any background task.

24. Self-improvement without evaluator capture

Self-improvement is a required core runtime capability from M3, not an optional M6 feature. It concerns bounded policies, prompts, operators, context selection, retrieval settings, scheduling, and tested code changes. The loop is observe, measure, propose, shadow-evaluate, compare, approve, version, deploy, monitor, and roll back. Production changes cannot be promoted by the same unprotected evaluator that the candidate may edit.

A candidate improvement must state its intended benefit, affected failure classes, relevant benchmarks, resource impact, and rollback path. It runs against frozen replay suites and fresh or sealed tasks, with the same budget and tool access as the incumbent. It must not exclude inconvenient failed missions from the denominator.

The improvement objective includes externally verifiable usefulness, scientific validity, calibration, diversity, cost, latency, and false-promotion risk. Lower token use at the expense of missed contradictions is not an improvement unless the mission explicitly accepts that trade-off and the change passes its guardrails.

Maintain champion and challenger versions. A promoted change remains attributable to its code, prompt, model, schema, and policy versions. Monitor drift and retain a last-known-good rollback target. Migration failures must not corrupt prior evidence.

Routing or retrieval changes require calibrated holdout tests appropriate to their role. A new model that ranks candidates differently is not automatically better because it gives its own candidates higher scores. A change to the evaluator itself requires a separate protected evaluation and explicit authorization.

The harness may learn from failure records and operator yields. It may not increase its spending limits, tool permissions, domain risk allowance, disclosure scope, or authority to rewrite its own gates. Those remain owner-controlled.

Mandatory autonomous loop [R-115–R-119]. Under standing scoped authorization, the runtime MUST originate improvement hypotheses from its own evidence, run bounded champion/challenger evaluations on fresh protected tasks, deploy qualified permitted local changes, persist and use learning on later missions, monitor canaries and restore verified incumbents on regression. Suggestions alone do not satisfy this requirement. `docs/SELF_IMPROVEMENT.md` defines targets, triggers, budgets, the typed improvement record, deployment gates and the M3 end-to-end demonstration. Permission, budget and evaluator changes remain owner-controlled; inconclusive cycles retain the incumbent.

Report self-improvement as a scoped measured result: which tasks, what budget, which metrics, uncertainty, and known limitations. Do not claim that a sequence of internal revisions demonstrates general scientific intelligence.

25. Evaluation framework and proof of value

Evaluation must test the entire chain, including discovery from a broad goal. A harness that performs well only when given a strong human hypothesis fails the central product requirement.

Use four layers. **Contract tests** establish structural and execution invariants. **Synthetic discovery worlds** provide hidden but known causal mechanisms, noise, confounders, and changed boundary conditions. **Retrospective rediscovery tasks** test whether the system reconstructs known ideas under a documented source cutoff. **Prospective bounded invention campaigns** test real utility and replication on new tasks with outcomes unavailable to the system beforehand.

Retrospective tasks have model-pretraining contamination risk even when the retrieved corpus is date-restricted. Report this limitation and avoid treating rediscovery as proof of novelty. Synthetic test generators and sealed workloads must be inaccessible to the invention workers. Prospective tasks are necessary for stronger claims of autonomous origination.

Baselines include a single capable model with active retrieval, a fixed generate-review loop, random or simple structured search where meaningful, and a known domain optimizer or existing method. Match resource budgets, model access, permitted data, and experimental tools. Run ablations without the graph, without adversarial review, without active discovery, without compiled task decomposition, and without optional typed decision models.

Core metrics are opportunity validity, evidence-qualified hypothesis yield, meaningful discriminability, source-span correctness, rediscovery detection, unique mechanism diversity, valid experiment rate, false-promotion rate, reproducibility, useful negative-result yield, verified engineering improvement, cost per qualified or validated outcome, and elapsed time including unsuccessful runs.

Model-judge scores are auxiliary. Blind expert review and independent objective tests supply stronger outcome measures where available. AgentIdeaBench is a useful precedent for distinguishing static reference input from active evidence exploration; its reported exploration gain did not imply higher measured originality under its critics. [S06]

Before a superiority claim, preregister a campaign, primary metric, guardrails, comparisons, and analysis. Determine campaign size from variance and detectable effect, not an arbitrary round number. As a pilot, 20 diverse missions may expose failure modes but does not establish statistical superiority. A release may ship as experimental after engineering gates pass even when the scientific value proposition remains unproven; the label must say so.

First campaign [R-102–R-105]. docs/CONTEXT_ASSEMBLY_CAMPAIGN.md defines a restricted Python import-closure domain, independently qualified oracle, tuned incremental baselines, repository-level sampling, complete denominators/costs, fresh change episodes and a fixed-sample analysis family. It separates campaign useful-outcome yield from negative-result yield and cost. The initial 20% latency/10% memory targets and 1% controlled false-promotion guardrail are provisional decisions. Sample size requires a recorded power/precision calculation before dispatch; no measured value is supplied by this package. AutoDiscovery is an additional precedent for system-chosen questions, but its model-belief surprise measure is not adopted as a scientific posterior or universal value metric. [S12]

26. Operational targets and acceptance gates

Targets below are provisional engineering budgets to be measured on a recorded reference machine. They are not achieved results. Pure state and policy operations should stay below 10 ms p95 under the specified local test load. A simple cached graph or text lookup should target below 100 ms p95 for the benchmark corpus. TUI input and progress acknowledgement should remain responsive without waiting for a model call.

Benchmark scheduler overhead separately from provider, network, computation, and experiment time. Target less than 10% added orchestration wall time for the defined synthetic workload, comparing against the same operations executed by a minimal runner. This target is not meaningful for arbitrary sub-millisecond tasks; such work should be batched rather than used to claim failure or success selectively.

Every release candidate must pass schema and migration tests, cross-record integrity checks, deterministic replay of recorded operations, concurrency budget enforcement, crash recovery, denied side-effect tests, protected-evaluator tamper tests, and source-provenance integrity tests. No critical issue may remain open for the scope being released.

Scientific gates are separate: false hypotheses and confounded effects must not be promoted in the controlled evaluation beyond the predeclared statistical error envelope; inconclusive data must stay inconclusive; incorrect units and ungrounded premises must be detected or explicitly unresolved; protected confirmation data must remain inaccessible to generation.

Performance acceptance uses end-to-end quality-adjusted outcomes. A faster configuration that loses valid hypotheses or weakens confirmation fails unless it remains within a predeclared noninferiority guardrail. Numeric speed targets do not override safety or scientific integrity.

Zero failures in a finite test suite is not evidence of zero real-world risk. Reports include the number of independent cases, failure counts, scope, and uncertainty. Release status is one of EXPERIMENTAL, QUALIFIED_FOR_DECLARED_SCOPE, or BLOCKED, with the scope and unresolved limitations attached.

The requirements register maps every normative requirement to an owner, milestone, and acceptance test. Runtime test cases in the package are specifications until an implementation runs them and records receipts.

Scope and clause coverage [R-113, R-114]. Every obligation has a clause anchor, enforcement owner/service, contract milestone, runtime milestone and positive/negative acceptance cases in the obligation matrix and register. `release-scopes.json` explicitly enumerates gates for each milestone; M0 contract readiness cannot claim an M3 end-to-end pass. Traceability verification checks reciprocal mappings, anchors, scope coverage and generated-document consistency. It does not prove that a qualitative test establishes every scientific premise. Remaining method/oracle/security qualification is release-blocking for the affected scope.

27. Implementation sequence and release strategy

M0: Contracts and evaluator foundations. Establish the repository, schemas, typed domain records, source provenance, threat model, requirements traceability, and protected fixture evaluators. Exit with deterministic package and contract checks; no claim of autonomous invention.

M1: Durable local execution. Build policy, budgets, task DAGs, artifact store, events, isolated workers, cancellation, and crash recovery. Exit with replay and fault-injection gates. Do not start with a swarm.

M2: Autonomous discovery and hypothesis generation. Add mission compilation, retrieval, opportunity mining, the initial operator library, search archive, hypothesis compiler, and prior-art triage. Exit with domain-only inputs producing operational hypotheses in hidden synthetic worlds, without seeded target mechanisms.

M3: Complete computational experiment and self-improvement loop. Add immutable plans, reference analysis, candidate realization, raw data, protected evaluation, outcome semantics, and dossiers. Exit with supported, contradicted, inconclusive, invalid, and blocked scenarios handled correctly end to end, plus autonomous bounded improvement, persisted subsequent use and rollback under protected gates.

M4: Quality and usability qualification. Add TUI/gateway polish, stronger retrieval, blind review support, reproduction, baseline campaigns, and first prospective use. Exit as experimental or qualified for the declared software scope according to measured evidence, not product ambition.

M5: Optional acceleration and advanced search. Add Jev or other decision adapters, Tachyon execution, graph-specific backends, GPU workers, learned allocation, and richer search only when ablations justify them. A failed optional integration must not break the default path.

M6: Additional domains and advanced learning. Add domain packs, supervised external protocols, optional model fine-tuning and advanced improvement strategies, and external replication. The mandatory self-improvement loop is already an M3 gate. Each new domain has its own validity and safety gates; software success is not inherited by physical science.

Parallel development is allowed for modules with stable contracts and independent tests. The critical path is contracts, durable execution, hypothesis validity, experimental integrity, and then throughput optimization. `IMPLEMENTATION_PLAN.md` provides task-level dependencies and acceptance deliverables.

28. Worked example: context assembly research

This is an illustrative design exercise, not an experimental result or a claim of novelty. The mission is to reduce time and cost of constructing correct coding-agent context while retaining complete relevant evidence under a declared task distribution.

Discovery inspects authorized traces and observes a candidate pressure point: repeated context construction may redo work after small repository changes. The system must first verify that the repeated work is material, that current caching is a credible baseline, and that “correct context” has an operational oracle. A full-rebuild straw baseline alone is inadequate.

An initial mechanism candidate caches context by filename. Adversarial review finds stale dependencies after imported modules change. This is a concrete failure, not simply a low score. A second candidate uses exact dependency invalidation and content hashes. Prior-art review is likely to find related incremental-build and cache-invalidation techniques; until searched, the status is unresolved, and a matching implementation would be a useful rediscovery rather than a new invention.

A further speculative candidate selectively refines dependency tracking for high-change regions while conservatively falling back to full reconstruction when dependency coverage is uncertain. The harness must specify how uncertainty is detected, why the fallback preserves the required correctness, and where refinement costs could exceed savings. Calling the policy “adaptive” is not sufficient.

The mechanistic prediction is that selective refinement reduces redundant reconstruction work in a defined workload regime without omitting oracle-required dependencies. The engineering target might provisionally require a 20% improvement over the strongest implemented baseline with no observed oracle violations in the declared tests and a separately justified reliability bound. The 20% target is an illustrative product choice, not a forecast.

The experiment compares full reconstruction, exact dependency caching, the candidate, and an ablation without selective refinement. Hold out repositories and change patterns. Freeze the context oracle and test manifests outside worker access. Measure end-to-end time, dependency coverage, peak memory, bookkeeping cost, and failure behavior under adversarial dependency changes. Randomize order and account for within-repository correlation.

Possible conclusions are deliberately different: the claimed mechanism contributes but the target is not met; the target is met but an ablation undermines the mechanism explanation; the candidate violates correctness; the study is underpowered; or a scoped benefit survives confirmation. Only the last relevant evidence path can support a validated-candidate label, and none establishes global novelty without the separate prior-art assessment.

29. Risks, open questions, and decision discipline

The principal product risk is that the system generates well-written, testable, already-known or unimportant ideas. Countermeasures are real-need discovery, claim-level prior-art checks, prospective tasks, and blinded evaluation of usefulness. Strong prose cannot compensate for weak novelty or value.

The principal scientific risks are hallucinated premises, invalid causal inference, correlated evidence, evaluator leakage, adaptive testing without control, poor simulation fidelity, and mistaking a failed implementation for a refuted mechanism. The principal runtime risks are uncontrolled fan-out, ambiguous side effects, budget race conditions, unsafe generated code, and unrecoverable state.

Unresolved research questions include whether a graph improves yield beyond a strong retrieval baseline; whether mechanism operators produce meaningful diversity; whether micro-task compilation saves total time after verification; whether typed decision models retain valuable unusual ideas; and whether the portfolio policy predicts useful outcomes. These are hypotheses about the harness itself and must be tested with ablations.

The first architecture decisions are intentionally conservative: software-first scope, modular monolith, provider-neutral contracts, optional Jev and Tachyon, protected evaluators, explicit inconclusive states, bounded search, and immutable experimental records. Changing them requires an architecture decision record with evidence and migration consequences.

The review register in `REVIEW_AND_RISKS.md` records an author-led multidisciplinary design review and the specific corrections made while drafting. It is not an independently convened expert board, a guarantee of completeness, or an achieved confidence score. Implementation findings can reopen any decision.

The master spec is sufficient to begin staged engineering without pretending that discovery performance is already known. The first consequential milestone is not an impressive UI or a thousand agents. It is a domain-only mission that generates its own worthwhile hypotheses, tests them honestly, preserves negative results, and reproduces the result under an independent evaluator.

Review remediation. Version 1.1 addresses the dated review with new qualification, lineage, inference-family, campaign, retrieval, memory/security, taxonomy and traceability obligations R-094–R-114. The archived probes remain reproducible against v1.0. New positive/adversarial synthetic tests exercise reference enforcement without asserting runtime receipts, authentic approvals or scientific results. The application remains unimplemented.

30. Source register and corrected research grounding

The following sources were checked on 30 September 2026. They are background and design precedents, not a complete literature review. Source descriptions intentionally avoid importing performance claims into the specification.

[S01] Google Research — Accelerating scientific breakthroughs with an AI co-scientist. 19 February 2025. Supports specialized generation/review/evolution roles and goal-driven hypothesis proposals; reports limitations and expert-guided validation. The earlier conversational description of this as a newly introduced 2026 system was inaccurate. <https://research.google/blog/accelerating-scientific-breakthroughs-with-an-ai-co-scientist/>

[S02] Google DeepMind — AlphaEvolve: A Gemini-powered coding agent for designing advanced algorithms. 14 May 2025. Supports evolutionary code search connected to automated evaluators in computational settings. <https://deepmind.google/blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/>

[S03] Huang et al. — Automated Hypothesis Validation with Agentic Sequential Falsifications. ICML / PMLR 267, 2025. Introduces POPPER and an explicit sequential-testing framework; its guarantees depend on its statistical construction and assumptions. <https://proceedings.mlr.press/v267/huang25n.html>

[S04] Rabby et al. — Iterative Hypothesis Generation for Scientific Discovery with Monte Carlo Nash Equilibrium Self-Refining Trees. arXiv:2503.19309, initially submitted 25 March 2025; a journal version is associated with DOI 10.1016/j.ins.2026.123576. Supports structured search as a hypothesis-generation precedent. The earlier “published this week” characterization is not used. <https://arxiv.org/abs/2503.19309>

[S05] Graph2Idea: Retrieval-Augmented Scientific Idea Generation with Graph-Structured Contexts. arXiv:2606.09105, 2026 preprint. A graph-context design precedent; use of a graph here remains an ablation question. <https://arxiv.org/abs/2606.09105>

[S06] Mo et al. — AgentIdeaBench: Benchmarking Scientific Ideation in the Agent Era. arXiv:2609.07611, submitted 7 September 2026. Compares static observation and active exploration. Its abstract attributes exploration gains to grounding, feasibility, clarity, and specificity rather than measured originality under its critics. <https://arxiv.org/abs/2609.07611>

[S07] TypeSafe — Introducing System One Models & Jev. 15 September 2026. Describes typed probabilistic model decisions. Provider claims about speed and calibration are not assumed to transfer to this workload. <https://typesafe.ai/blog/introducing-system-one-models-and-jev>

[S08] razorback16/openjev — project repository. Repository documentation accessed 30 September 2026. An example third-party decision server, not a required implementation or proof of compatibility with every project named OpenJev. Pin and test any selected adapter. <https://github.com/razorback16/openjev>

[S09] W3C — PROV-Overview. 30 April 2013. Provides provenance concepts for entities, activities, and actors; used as an interoperability reference. <https://www.w3.org/TR/prov-overview/>

[S10] Howard et al. — Time-uniform, nonparametric, nonasymptotic confidence sequences. arXiv:1810.08240; originally submitted 18 October 2018. A primary reference for time-uniform confidence-sequence methods, not a universal drop-in test. <https://arxiv.org/abs/1810.08240>

[S11] JSON Schema — Draft 2020-12. Specification documentation accessed 30 September 2026. Defines the schema dialect used by the interchange contracts. <https://json-schema.org/draft/2020-12>

[S12] Agarwal et al. — AutoDiscovery: Open-ended Scientific Discovery via Bayesian Surprise. NeurIPS 2025; arXiv:2507.00310v3 revised 12 February 2026. A precedent for system-chosen questions and search guided by

model-belief surprise, not proof of calibrated posteriors or practical invention utility. <https://arxiv.org/abs/2507.00310v3>

[S13] Xiong et al. — AutoResearchBench: Benchmarking AI Agents on Complex Scientific Literature Discovery.
arXiv:2604.25256, submitted 28 April 2026. Separates deep target-paper discovery and wide literature collection; a scoped retrieval-qualification precedent. <https://arxiv.org/abs/2604.25256>

[S14] Gadgil et al. — Bad Memory: Evaluating Prompt Injection Risks from Memory in Agentic Systems.
arXiv:2607.14611, submitted 16 July 2026; preprint. Studies persistent instruction attacks across sessions in evaluated agent systems; motivates memory trust propagation without universal attack-rate claims. <https://arxiv.org/abs/2607.14611>

[S15] Anthropic — Improving our alignment and security practices. August 2026, primary organizational account. Describes layered sandbox validation and monitoring in high-risk evaluation settings; those settings differ from this computational invention scope. <https://www.anthropic.com/news/improving-alignment-security-efforts>

31. Final readiness contract

The intended experience is: authorize a bounded goal, let the harness discover the questions, inspect mechanisms and competing explanations, run permitted tests, and receive an evidence-backed outcome. The user must not have to invent the hypothesis for the invention system.

Completion is not “the agents agree.” It is a typed, auditable result with the appropriate evidence, artifact, uncertainty, novelty scope, and authorization history. An honest inconclusive result can complete a bounded investigation; a confident unsupported claim cannot.

The implementation must preserve the distinction between an exploratory idea, a test-ready hypothesis, a supported scoped claim, a useful prototype, a validated invention candidate, and a commercially or legally assessed invention. Each step has its own evidence and authority requirements.

Build the smallest end-to-end version first. Prove that it discovers and tests its own hypotheses more usefully than the matched baselines. Then add search sophistication, parallelism, specialized models, new domains, and self-improvement only where the evidence justifies them.

Appendix 1: Qualification and lineage

Version 1.2 · 30 September 2026 · Normative supplement to master sections 13–16, 20–21, and 26. Requirements R-094–R-101 and R-112–R-114 apply. This defines future runtime obligations; included Python checks exercise selected metadata only.

Protected trust context [R-094, R-095]

Candidate-supplied `valid`, `supported`, `qualified`, `approved`, and `independent_pass` fields MUST NOT establish authority. A protected service MUST authenticate receipt/grant/qualification issuers, validate their immutable signed or authenticated contents and revocation state, retrieve artifacts and verify their bytes, and establish current dependency and policy versions. Importing a record or setting its `trust_origin` is not authentication. Receipts MUST be bound to exact record versions and candidate, evaluator, analysis, evidence-snapshot, and policy identities. A digest without accessible verified bytes is insufficient. Each receipt also binds the exact assessed result/dossier payload and plan registration digest. Assessment payload hashing excludes only receipt-reference fields to prevent hash cycles; altering findings, conclusions or labels requires a new assessment.

`ValidationContext` in the reference code is an externally supplied test seam. It MUST NOT be populated from model output, the candidate bundle, or a user-editable cache. Trusted identity sets also require matching externally authenticated record-content hashes; freshness binds both version and immutable content. Passing synthetic context demonstrates metadata behavior, not real authentication or qualification. The runtime MUST reject fixture trust contexts in live promotion. The reference default is empty trust and therefore cannot promote a candidate. The schema includes five additional records: authorization grant, method qualification, experiment family, holdout access, and verification receipt. Each is an immutable domain record; only the protected authority can confer trust on it.

Complete qualification path [R-094, R-095, R-096]

For every claim and mission guardrail needed for a label, the promotion service MUST resolve the exact hypothesis and executable plan, ensure blockers are resolved, and verify the applicable method qualification. It MUST check primary endpoint coverage, units, failed or missing controls, guardrails, raw artifacts, analysis artifacts, deviations, independent reproduction, and the final promotion assessment. Execution success alone does not establish a supported claim. Failed controls invalidate the affected inference; failed engineering guardrails can leave a mechanism informative while blocking the solution label.

Fixed-sample empirical qualification requires its registered design and sample-size rationale. Sequential qualification requires an independently qualified stopping method and family policy. Formal qualification requires the proposition, assumptions, proof/counterexample artifacts, and a trusted checker; deterministic qualification requires a declared exhaustive or bounded test and oracle scope. Formal or deterministic work uses the `nonstatistical` family and MUST NOT invent statistical sample sizes, alpha or confidence intervals. Registration freezes its proof/test obligations and the access ledger records protected evaluator-input access. Every design has immutable method/evaluator identity and explicit limitations.

The v1.1 reference profile deliberately checks a single hypothesis and qualifying result for a dossier, with frozen confirmatory registration, trusted receipts, endpoint/control/guardrail coverage, and externally verified dependencies. Unsupported multi-hypothesis promotion fails closed. Runtime composition can support additional profiles only after their claim aggregation and independence rules pass contract tests. Independent reproduction requires a protected assessment of personnel, generator, measurement pipeline, artifacts, and data dependencies; merely changing an issuer name does not establish independence.

Lineage and applicability [R-097, R-098]

Relationship	Required binding	Change consequence
Opportunity → mission	Same immutable mission version	New mission requires applicability assessment
Mechanism → opportunity	Exact opportunity version	New mechanism gets new lineage
Hypothesis → mechanism/opportunity/mission	Coherent chain; no cross-mission substitution	Recompile and reassess applicable evidence
Plan → hypothesis	Exact hypothesis, predictions, candidate, comparator, evaluator, analysis, snapshot	Material change requires new registered version
Result → plan/hypothesis	Exact execution and analysis identities, endpoint and unit coverage	Cannot relabel old observations as a new run
Dossier → hypotheses/results	Every result addresses a listed hypothesis in the dossier mission	Cross-mission reuse requires explicit applicability assessment
Grant → operation	Exact mission, operation, destination, artifact, policy, cost and expiry	Changed identity or revoked grant blocks dispatch
Cache/summary → inputs	Semantic task, source, record, prompt, model/tool, policy and environment dependencies	Relevant change invalidates reuse

The runtime **MUST** maintain dependency edges from evidence, sources, grants, qualification, evaluator, and artifacts to conclusions. Corrections, retractions, quarantine, changed policy, and changed scope **MUST** mark affected current labels stale and block reuse pending assessment. Historical records remain intact. New versions **MUST NOT** inherit old approval or support automatically. An explicit signed applicability assessment can approve historical evidence reuse within named conditions; absent that supported contract, the v1.1 reference rejects stale/unknown versions.

Authorization timing [R-099]

Authorization **MUST** be checked at compilation, immediately before dispatch, and before externally consequential effects. Grants include issue/expiry time, revocation state, operation, destination, artifact, capabilities, policy, and maximum cost. Mission capabilities and destination allowlists also apply. A standing local profile may mint scoped grants deterministically without a new user prompt. Time must come from the control plane, not the worker's creation timestamp. Concurrent cost reservation remains a separate transactional prerequisite.

Revocation stops new dispatch and effects; already completed effects are reconciled rather than represented as undone. Cancellation, checkpoint, and reconciliation behavior **MUST** be specified per capability. Dependency writes and redirects cannot bypass destination scope. Policy changes invalidate outstanding grants unless an explicit approved compatibility rule applies.

Registration and access [R-100, R-101]

Registration binds all plan fields except the operational status and the registration envelope itself. v1.1 defines the hash profile as UTF-8 JSON with sorted object keys, compact separators, unescaped Unicode, and finite numbers, as implemented by `plan_digest`. Implementations **MUST** use shared canonicalization vectors; different JSON encoders or numeric representations must not silently produce equivalent-looking payloads. The schema version binds this profile. Any hash-profile migration requires a new version and cannot rewrite historical registrations.

The future service **MUST** freeze the plan before results/holdout access, make registered records immutable, and authenticate the registration event. The reference recomputes the payload digest but cannot authenticate its history. A sealed-data broker **MUST** create immutable access events and track feedback queries, family membership, selection lineage and data partitions across missions. Worker-supplied timestamps do not establish absence of prior access.

The initial campaign uses `bonferroni_fixed_family`: no more than the registered number of confirmatory endpoint tests across all member plans, family-wise alpha divided across that fixed family, and no detailed feedback until campaign closure. Related claims and guardrails use the predeclared joint error policy. Independent fresh confirmation may address selection on exploratory data only when independence is established. Family resets, renamed missions, and dataset aliases **MUST NOT** reset exposure. Sequential methods and adaptive holdout reuse are unavailable until separately qualified. Retired/overexposed data cannot confer confirmation; the next campaign uses fresh independent workloads.

Label taxonomy [R-112]

`completed_investigation` includes informative negative and inconclusive outcomes. `validated_solution` requires scoped scientific/engineering/guardrail/reproduction qualification and a resolved scoped prior-art assessment; known prior art is allowed and displayed prominently as rediscovery. `validated_candidate` adds a protected claim-level difference assessment and verified claim-chart artifact; it permits `near_match` or `no_match_within_search_scope` only within the stated coverage. Neither implies global novelty or a legal conclusion. `known` is not eligible for the invention-candidate label. Scientific conclusions stay identical when only the prior-art label changes.

Explicit state transitions [R-114]

Record	Allowed progression	Blocking/revision rule
Opportunity	discovered → grounded → prioritized → explored → parked/closed	Reopening creates a version and reason
Hypothesis	exploratory → compiled → reviewed → test_ready → testing → assessed	Blocked/archive transitions preserve evidence; material revision restarts affected gates
Experiment plan	draft → ready → running → completed	Blocked plans cannot run; frozen payload cannot change
Task	pending → ready → running → completed/failed/cancelled	Retry creates a new attempt under the same declared effect semantics; unknown effects reconcile
Dossier	exploratory/planned → completed_investigation → validated_solution or validated_candidate	Promotion requires current protected assessment; invalidation creates a stale assessment event

Transitions are validated control-plane commands with expected prior version, event ID, actor, and reason. Each transition has authorization, reservation, evidence and output prerequisites. No transition overwrites a record version. Migration **MUST** reject unsupported schema versions, preserve raw old records and hashes, and emit a migration receipt with old/new identities and semantic differences. The archived v1.0 schema/fixture are compatibility inputs, not a live 1.1 bypass.

Appendix 2: First domain and campaign

Version 1.2 · 30 September 2026 · Normative supplement to master sections 25–28. Requirements R-102–R-105 apply. This is a protocol to implement, not a measured result or an implemented evaluator.

Narrow executable domain

The first domain is incremental assembly of coding context for **statically resolvable Python module dependencies**, rather than an undefined oracle for all semantic relevance. A domain-only mission asks the system to reduce correct context-assembly cost; workers receive authorized repositories, observations and tool contracts without a target mechanism. The harness still must originate its opportunities and hypotheses. The domain pack supplies measurement semantics and guardrails, not a desired solution.

Eligible repositories have valid parseable Python modules, an explicit import-resolution manifest, reproducible local dependencies and license/access permission. Exact versions of repositories and change episodes are recorded. Required context is the conservative transitive closure under the declared import-resolution rules plus the changed roots. Relative imports, import cycles, package initializers, aliases, deletions and renames are included. Unsupported dynamic imports, generated modules or ambiguous resolution must trigger conservative full reconstruction or a typed unsupported-case outcome; they cannot silently omit dependencies. This operational oracle does not claim all information needed for arbitrary coding tasks.

A protected reference oracle recomputes from the complete input snapshot with an independently maintained implementation. Hand-adjudicated fixtures, metamorphic tests and a second implementation qualify it before experiments. Record oracle disagreements, adjudication and scope limitations. Generator workers cannot read oracle internals, sealed workloads or future change episodes. The baseline authors cannot choose workloads after seeing candidate results.

Baselines and workload registration

Baselines **MUST** include (1) full reconstruction for correctness and total-cost accounting; (2) a strongest available tuned exact dependency/content-hash caching implementation; (3) a single capable model with active retrieval and identical experiment tools; and (4) a bounded generate/review or structured-search harness. An ablation removes the candidate's claimed causal ingredient. A full-rebuild-only comparison cannot establish a useful incremental improvement.

The campaign manifest **MUST** pin implementations, tuning allowance, model/version/tool access, corpus snapshots, hardware/OS, measurement instrumentation, and repository episode generator. Sample repositories independently from a documented eligible frame, stratify by size and dependency topology, and sample change episodes before confirmation. Train/tune, pilot, and confirmatory partitions are separated by repository; deliberate distribution-shift stress cases are reported separately. A curator prepares fresh future change episodes after candidates are frozen. Reused public repositories retain pretraining-contamination limitations; fresh episodes mitigate only some contamination.

All harness arms get the same approved compute/retrieval/experiment envelope and standing local permissions. Baseline tuning and engineering time are recorded. Repeated model seeds/trajectories within one repository are not independent missions. Costs include acquisition, generation, failed/invalid/blocked runs, evaluator and reproduction work, and human intervention; record both provider charges and resource/human-time quantities rather than inventing a conversion price. Report paired repository-level comparisons and dependence assumptions.

Metrics and numerical decisions

Metric	Operational definition	Denominator and interpretation
Context correctness	Required oracle modules present with correct snapshot identity	All attempted eligible episodes; omissions and stale context are failures
Opportunity validity	Independently confirmed material bottleneck or need	All proposed opportunities, with unresolved cases retained
Qualified hypothesis yield	Distinct hypotheses passing named readiness gates	All generated hypotheses; similarity adjudication and lineage recorded
Useful outcome yield	Independently reproduced prototype meeting the frozen correctness, latency and resource gates	All assigned missions, including failed/blocked/inconclusive; negative-result yield reported separately
End-to-end cost/time	Total acquisition through confirmation/reproduction cost and elapsed time	Every assigned mission; zero qualified outcomes means cost-per-outcome is undefined, not zero
Mechanism contribution	Predeclared candidate versus causal ablation/comparator observations	Repository-level units; separate from engineering attainment
False promotion	False/confounded controlled cases incorrectly receiving a scoped validated label	All independently sampled controlled cases, stratified by failure type
Reproduction	Successful clean-environment independent assessment	Every reproduction attempt, with environment/measurement dependencies visible

For the **initial engineering candidate**, the provisional target remains at least 20% end-to-end latency reduction relative to tuned exact caching, with zero observed context omissions/staleness, and peak memory no more than 10% above that baseline. The system measures initialization, invalidation, lookup, bookkeeping, fallback and amortization under the declared episode horizon. Both percentages are owner-reviewable provisional product choices, not empirical predictions or justified sample sizes. An interval crossing a target is inconclusive. No observed correctness failures still requires a reported reliability bound; it is not proof of perfect correctness. Formal correctness under the declared import model, where available, is separate evidence.

The **primary campaign comparison** is the difference between arms in the fraction of all assigned missions yielding a useful qualified outcome. Superiority requires the predeclared multiplicity-adjusted lower confidence bound for the paired difference to exceed zero while correctness and validity guardrails pass. Report outcome yield and total cost side by side; cost per outcome is secondary, and negative-result usefulness is a separate measured output. A higher useful-outcome rate at a higher cost is not automatically economic superiority.

M3 implements one fixed-sample method: paired repository-level comparisons, a bounded-outcome concentration interval for latency savings and binary mission-yield differences, with alpha allocations from a fixed Bonferroni family. Before runtime qualification, a statistician or protected method qualification must establish clipping/bounding policy, estimands, independence conditions, treatment of aborted runs and missing observations, and valid noninferiority analysis. Bounds/clipping are frozen before confirmation and clipped rates are reported; alternative robust or hierarchical methods require separate qualification. Failed or missing runs remain failures for useful-outcome yield; missing timing data cannot become a favorable latency estimate. Any unqualified assumptions block confirmation rather than invite model improvisation.

The campaign manifest chooses n from a separate exploratory pilot or defensible variance/precision bounds and the smallest effect meaningful to the owner. It **MUST** include the calculation, alpha allocation, reference distribution, power/precision target and sensitivity assumptions. No runtime manifest with `n=null`, an unqualified method, or an unresolved oracle can enter confirmation. The package deliberately supplies no fabricated sample-size estimate.

For the initial controlled false-promotion qualification, a provisional guardrail is a one-sided 95% upper rate bound no greater than 1% on a declared independent controlled-case distribution, with additional stratified stress checks. Under IID Bernoulli

cases and zero failures, the exact bound is $1 - 0.05^{**}(1/n)$; $n=299$ would satisfy the numerical criterion. This is a planning calculation, not a prescribed sample size for heterogeneous cases or a universal safety claim. Dependence, adaptive selection and distribution mismatch require a qualified alternative. Any evaluator-tampering failure blocks release regardless of aggregate rate.

Release dispositions and stopping

M0/M1 engineering releases require applicable contract/isolation/recovery gates, not scientific superiority. M2 demonstrates domain-only discovery on sealed synthetic worlds. M3 handles all outcome classes with trusted artifacts and method-specific checks. M4 may be EXPERIMENTAL after engineering gates without demonstrated value. QUALIFIED_FOR_DECLARED_SCOPE requires the frozen campaign, protected oracle/method qualification, error and correctness guardrails, independent reproduction, and scope-specific security gates. A superiority claim additionally requires the primary comparison above. The campaign closes on its registered sample/resource/elapsed limits, not when a favorable result appears. No useful outcome is a valid campaign result.

Appendix 3: Retrieval and memory security

Version 1.2 · 30 September 2026 · Normative supplement to master sections 6, 10, 16, 18, 20 and 25. Requirements R-106–R-111 apply.

Retrieval qualification

Before retrieval supports readiness or novelty assessment, adapters **MUST** be tested on both deep target-paper discovery and wide relevant-set collection. Cases include difficult synonyms, equivalent mechanisms in adjacent domains, citation chains, inaccessible methods, known near matches, corrections/retractions and counterevidence. Reference-set completeness and independent adjudication **MUST** be documented. Precision/recall applies only to that bounded reference set; it cannot be called global search coverage.

Critical claim retrieval **MUST** preserve locator, content hash, parser/version, span coordinate system and transformations. A span is verified against its captured source bytes, not a current webpage with the same URL. OCR or normalization discrepancies remain explicit. Deep/wide success, counterevidence recall, quote/span correctness, inaccessible-source rate and near-match detection are separate metrics. The mission's readiness policy states which missing evidence blocks qualification. Search services cannot certify their own completeness.

Approved proprietary search queries **MUST** be checked at the network boundary against disclosure scope, including expanded queries, redirects and follow-up fetches. An authorization to read public literature does not authorize leaking a mechanism in the request. Query/coverage receipts and claim charts must be preserved for each novelty assessment. No no-hit query creates a global novelty or legal claim.

Trust propagation and poisoning

Derived summaries, embeddings, graph edges, caches, failure memories and operator statistics **MUST** retain input provenance and trust origin. Untrusted content remains untrusted through transformation; an LLM summary cannot convert retrieved instructions into owner policy. Behavioral/policy memory is writable only through explicit owner-controlled change operations. Models can propose bounded changes but cannot promote them to authoritative instructions.

Ingestion **MUST** offer quarantine with recorded reason and dependencies. Quarantine, source correction/retraction, or discovered malicious instructions invalidates affected current summaries/caches/labels and queues permissible review; immutable originals remain available for audit. Model memory cannot grant tools, revise endpoints, mint qualification, or conceal failed runs. Qualification uses a protected current dependency registry, not a worker's claim that a cache is fresh.

Required security tests plant harmless adversarial instruction payloads in source documents and imported memories, run several sessions, and inspect instruction authority, memory writes, leakage and dependent-artifact invalidation. Test poisoning via summaries and graph edges as well as direct input. False-positive quarantine and loss of useful adaptation must be measured; blindly deleting all memory is not the desired defense.

Containment and monitoring

The first execution profile uses isolated nonprivileged workers with read-only snapshots, bounded writable output, denied network, explicit resource limits, and brokered tool access. The implementation decision **MUST** name its actual OS isolation mechanism and tested kernel/container/VM assumptions; subprocess alone is not isolation. Model/provider API access and credentials are mediated outside the worker environment.

Before each dispatch, the protected service **MUST** attest the effective mounts, identities, inherited file descriptors, subprocess restrictions, resource limits and network policy. Tests cover traversal/symlinks, private/loopback addresses, DNS and redirects, inherited sockets, dependency installation hooks, evaluator access, and denial of service where applicable. Changes in host configuration invalidate the attestation. Declared network grants are enforced at the broker, not only in prompts.

During execution, a protected monitor **MUST** detect and stop tested boundary violations, preserve raw events, quarantine outputs and revoke affected sessions/grants. A deterministic boundary mechanism remains authoritative; model-based anomaly detection is additional advisory coverage and cannot guarantee containment. Monitoring, artifact collection and logs must not leak secrets. Revocation and cancellation races are tested before an autonomous-execution release.

Appendix 4: Required autonomous self-improvement

Version 1.2 · 30 September 2026 · Normative supplement to master section 24. Requirements R-070–R-072 and R-115–R-119 apply. This is a required runtime capability from M3, not a promise that every iteration improves or a claim of implemented learning.

Autonomous improvement loop [R-115]

Hephaestus **MUST** improve its own research process using observations from completed, failed, invalid, blocked and inconclusive investigations. A bounded self-improvement service **MUST** observe, diagnose, propose, compile, shadow-test, independently qualify, deploy permitted changes, monitor and roll back. It starts from broad missions without a user-supplied improvement hypothesis. It **MUST** generate its own improvement hypotheses and use the same evidence discipline applied to inventions.

Within an approved standing improvement policy, the service **MUST** execute routine reversible local improvements without asking the owner to design each change or approve each iteration. The owner authorizes the scope, resource envelope and promotion policy; a deterministic protected service mints exact grants and decides whether evidence permits promotion. Merely generating suggestions, accumulating summaries or asking a human to implement every change does not satisfy autonomous self-improvement.

The mandatory M3 path improves at least prompts, hypothesis/operator selection, context compilation, retrieval settings and bounded scheduling/allocation configuration. It must also support proposing and sandbox-testing changes to worker/runtime code; deployment of code changes requires the applicable separately scoped capability and integration/migration/security checks. Code deployment can remain blocked while the approved configuration path operates. Model fine-tuning, new domains and distributed execution are later optional mechanisms; self-improvement is not postponed until they exist.

Triggers are recorded: mission completion, a predeclared batch/interval, independently measured drift, repeated failure class, or an owner request. Each trigger produces a bounded proposal or a recorded no-justified-change outcome. Each cycle has a reserved budget, concurrency/depth/candidate cap, deadline and stop rule. Recursive improvement proposals remain within the original envelope; the service cannot multiply budgets by spawning improvement missions. If no change qualifies, retain the incumbent and preserve the negative/inconclusive learning result.

Durable learning and change targets [R-116]

Persist mission outcomes, interventions, operator yields, rejection reasons, verified failures, corrections and champion/challenger identities with source and trust provenance. The service **MUST** use this history in subsequent candidate generation and routing, subject to confidentiality and stale/quarantine rules. Restarting cannot lose learned decisions or reset holdout exposure. A changed condition may reopen an old failure only through a new version with evidence of that changed condition.

Learning targets include better opportunity discovery, discrimination, source/counterevidence retrieval, hypothesis diversity, analysis-plan completeness, context efficiency, scheduling and implementation correctness. Neither an unverified memory nor a model rating is ground truth. Weight updates or learned routing outputs remain advisory unless qualified for their declared domain; deterministic authorization, budgets and promotion gates remain protected. No local fine-tuning or particular model is required for the system to improve.

Evidence and deployment contract [R-117]

Every `improvement_candidate` **MUST** identify the incumbent, challenger, target, supporting observation records, immutable evaluation manifest, fresh evaluation partition/family, total resource budget, intended primary benefit, guardrails, rollout and rollback artifact. The manifest freezes effect/noninferiority bounds, sampling units, complete unsuccessful-run denominators, tooling access, method qualification, stopping and multiplicity rules before accessing outcomes.

Compare champion and challenger at matched data, model/tool access and resource allowance. Frozen replay catches regressions; fresh protected tasks test generalization. Repeated challenger queries use the same protected family/exposure ledger rules as other confirmation. A benchmark/evaluator change is evaluated separately with explicit owner authorization and cannot certify its own candidate. Correlated agents or repeated timings are not independent confirmation.

Promotion MUST require a protected, authenticated assessment bound to the exact improvement payload, verified candidate/evaluation/rollback artifacts, a supported predeclared benefit, passing scientific/correctness/security guardrails and current scoped grant. Improving speed alone while weakening evidence, diversity, correctness or false-promotion control is not qualification. Inconclusive or failed evaluations do not deploy. A model cannot attest its own qualification, change the incumbent metric after seeing results, or hide failed candidates.

The assessment includes `benefit_status`, `guardrail_status` and evaluator identity. The reference code checks externally authenticated metadata; it does not compute statistical significance or authenticate an issuer. Live contexts reject synthetic qualification. The runtime must independently verify the actual study, family/exposure history, assumptions and integration evidence before establishing trusted improvement context.

Canary, recovery and rollback [R-118]

Deploy a qualified change first to a bounded canary/shadow scope specified in the grant. Capture code, prompt, model, policy, schema and data identities; update the champion pointer transactionally without overwriting the incumbent. Monitor predeclared quality, cost, latency, false-promotion, security and drift indicators. Violating a protected guardrail or a failed migration MUST stop the affected rollout and restore the verified incumbent under a standing rollback capability. Do not erase experiments or pretend completed side effects were undone.

Each rollout records its observed scope, qualified assessment, deployment receipt and rollback target. Restart and crash recovery MUST reconstruct the champion and reconcile interrupted deployments. A rolled-back candidate retains the reason and observations; retries need new evidence/version rather than automatic repeated rollout. A changed improvement policy or permission invalidates affected grants. Automatic deployment remains limited to actions covered by standing authorization.

Required demonstration and boundaries [R-119]

M3 MUST demonstrate an end-to-end controlled loop: a broad mission reveals a nonseeded reproducible failure/opportunity; the service proposes a challenger; protected fresh evaluation qualifies or rejects it; a qualified permitted change becomes the persisted champion; a later mission uses it; restart preserves the state; an injected regression triggers rollback. False, confounded, overbudget, permission-expanding and inconclusive challengers must remain undeployed. A generated patch or an edited prompt without these behaviors is insufficient.

Actual improvement claims report the evaluated tasks, baseline, budgets, effect/uncertainty, failed proposals, scope and remaining limits. An experimental runtime may ship with functioning self-improvement gates before general benefit is established; it cannot claim monotonically increasing intelligence. The system MUST NOT rewrite its safety policy, spending limits, disclosure scope, protected data, evaluator, analysis qualification or promotion authority through this loop. Changes to those boundaries remain owner-controlled and separately authorized.